VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Využití behaviorálních stromů v Unreal Engine 5

Impementing Behavioral Trees in Unreal Engine 5

Roman Kratochvíl

Bakalářská práce

Vedoucí práce: Ing. Jakub Beránek

Ostrava, 2026

VŠB TECHNICKÁ UNIVERZITA OSTRAVA | FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Č.j.VSB/24/131376
vsboee2403D4C8

Zadání bakalářské práce

Student: **Roman Kratochvíl**

Studijní program: B0613A140014 Informatika

Téma: Využití behaviorálních stromů v Unreal Engine 5
Impementing Behavioral Trees in Unreal Engine 5

Jazyk vypracování: čeština

Zásady pro vypracování:

Cílem práce je zanalyzovat, jak lze využít behaviorální stromy k implementaci chování nehrajících postav (NPC) ve frameworku Unreal Engine 5 (UE5), a demonstrovat jejich využití na vzorové hře. Práce by se měla zaměřit na využití behaviorálních stromů k implementaci zajímavého chování nehrajících postav, např. dynamická interakce s prostředím, reakce na chování hráče nebo aktivní spolupráce mezi více nehrajícími postavami. Práce by měla navrhnout a popsat několik typů chování a naimplementovat je ve vzorové hře, která bude demonstrovat, jak dobře interakce nehrajících postav fungují a jak reagují na chování hráče. Výsledný kód by měl být řádně zdokumentován a při jeho vývoji by měl být využit verzovací systém (např. git).

1. Analyzujte a popište principy behaviorálních stromů, a srovnejte je s jinými technikami implementace chování NPC.
2. Nastudujte možnosti UE5 v oblasti definice chování nehrajících postav.
3. Navrhněte několik typů netriviálního chování nehrajících postav využívající pomocí behaviorálních stromů.
4. Naimplementujte jednoduchou hru, na které tyto typy chování demonstrujete.
5. Zdokumentujte kód a k jeho vývoji použijte verzovací systém (např. git).

Seznam doporučené odborné literatury:

[1] Stuart Butler and Tom Oliver: Game Development Patterns with Unreal Engine 5: Build maintainable and scalable systems with C++ and Blueprint, 2024, Packt Publishing, ISBN: 978-1803243252
[2] Michele Colledanchise and Petter Ögren: Behavior Trees in Robotics and AI: An Introduction, 2018, CRC Press, ISBN: 978-1138593732
[3] Robert Nystrom: Game Programming Patterns, 2014, ISBN: 978-0990582908

17. listopadu 2172/15
708 00 Ostrava-Poruba
Česká republika

spojovatelka: +420 597 321 111
epodatelna: epodatelna@vsb.cz
ID datové schránky: d3kj88v

IČ: 61989100
DIČ: CZ61989100

email: studijni.fei@vsb.cz
www.fei.vsb.cz

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **Ing. Jakub Beránek**

Datum zadání: 01.09.2024

Datum odevzdání: 30.04.2025

Garant studijního programu: doc. Mgr. Miloš Kudělka, Ph.D.

V IS EDISON zadáno: 25.10.2024 13:36:49

Abstrakt

Tato bakalářská práce se zabývá využitím behaviorálních stromů pro implementaci chování nehrajících postav (NPC) v prostředí Unreal Engine 5 (UE5). Cílem práce je analyzovat principy behaviorálních stromů, porovnat je s jinými metodami implementace NPC a demonstrovat jejich praktické použití na vzorové hře. Práce se zaměřuje na návrh a realizaci netriviálního chování NPC, zahrnujícího například dynamickou interakci s prostředím, reakce na hráče a spolupráci mezi NPC. Součástí práce je dokumentace principů behaviorálních stromů, návrh několika typů chování, jejich implementace ve vzorové hře a využití verzovacího systému při vývoji.

Klíčová slova

behaviorální stromy, nehrající postavy, Unreal Engine 5, chování NPC, verzovací systém

Abstract

This bachelor's thesis explores the use of behavioral trees for implementing non-player character (NPC) behavior within the Unreal Engine 5 (UE5) framework. The aim is to analyze the principles of behavioral trees, compare them with other NPC behavior implementation techniques, and demonstrate their practical application in a prototype game. The thesis focuses on designing and implementing complex NPC behaviors, including dynamic interaction with the environment, responses to player actions, and collaboration among NPCs. The work includes documentation of the principles of behavioral trees, the design of several behavior types, their implementation in a prototype game, and the use of a version control system during development.

Keywords

behavioral trees, non-player characters, Unreal Engine 5, NPC behavior, version control system

Poděkování

Rád bych na tomto místě poděkoval všem, kteří mi s prací pomohli, protože bez nich by tato práce nevznikla.

Obsah

Seznam použitých symbolů a zkratek **8**

Seznam obrázků **9**

Seznam tabulek **10**

1 Úvod **11**

2 Behaviorální stromy **12**

2.1 Úloha behaviorálních stromů 12
2.2 Základní princip vykonávání 13
2.3 Struktura behaviorálního stromu 13
2.4 Základní typy uzlů 14
2.5 Výhody behaviorálních stromů 17
2.6 Omezení behaviorálních stromů 17
2.7 Srovnání s jinými technikami implementace chování NPC 18

3 Unreal Engine 5 a AI systém **20**

3.1 Herní engine a Unreal Engine 5 20
3.2 Pawn a AI Controller 20
3.3 Behaviorální strom v UE5 21
3.4 Blackboard 21
3.5 Navigační systém 21
3.6 AI Perception 22

4 Návrh a architektura NPC systému **23**

4.1 Přehled systému a jeho komponent 23
4.2 Rozdělení odpovědností 24
4.3 Stavový model NPC 25
4.4 Architektura behaviorálního stromu 26

4.5 Tok dat v systému . . . 28
4.6 Správa hlídkových tras . . . 28
4.7 Reprezentace znalostí NPC . . . 28
4.8 Komunikace mezi NPC . . . 30
4.9 Reakce na hráče a prostředí . . . 30
4.10 Návrhová rozhodnutí . . . 30

5 Implementace systému NPC **32**
5.1 Implementace BP_NPC . . . 32
5.2 Implementace AI Controlleru (IAC_NPC) . . . 34
5.3 Implementace stromu chování (BT_NPC) . . . 37
5.4 Implementace úloh a služeb . . . 38
5.5 Implementace hlídkových tras . . . 40
5.6 Sdílení informací o předmětech . . . 41
5.7 Implementační problémy a rozhodnutí . . . 41

6 Testování a evaluace řešení **42**

7 Závěr **44**
7.1 Možná rozšíření a budoucí práce . . . 44

Literatura **46**

Přílohy **46**

A Struktura projektu **48**

Seznam použitých zkratek a symbolů

UE5	–	Unreal Engine 5
NPC	–	Non-playing character
FSM	–	Finite State Machine
AI	–	Artificial intelligence
RTS	–	Real-Time Strategy

Seznam obrázků

2.1 Zjednodušená struktura behaviorálního stromu pro NPC. 16

4.1 Přehled komponent systému NPC a jejich vzájemných vazeb 24
4.2 Přehled hlavních stavů NPC a přechodů mezi nimi 26

5.1 Umístění události `AN_HitEvent` v animační montáži 33
5.2 Implementace události `AN_HitEvent` v blueprintu 33
5.3 Implementace metody CanSenseActor 35

1 Kompletní struktura behaviorálního stromu použitého v systému NPC 47

Seznam tabulek

2.1 Typy uzlů v behaviorálním stromě. Přeloženo podle [1, s. 9, tab. 1.1]. 16

6.1 Přehled manuálně ověřených scénářů 42

Kapitola 1

Úvod

Herní vývojáři se snaží vytvářet herní prostředí, které působí pro hráče co nejvíce uvěřitelně a interaktivně. Jedním z hlavních prostředků, jak tohoto cíle dosáhnout, jsou nehráčské postavy.

Nehráčská postava (z anglického *Non-Player Character*, dále zkráceně NPC) představuje herní entitu, která není přímo ovládána hráčem, ale jejíž chování je řízeno umělou inteligencí. NPC pomáhají řešit problém „mrtvého světa“, tedy prostředí, které nereaguje na hráčovy akce a neposkytuje odpovídající zpětnou vazbu.

S rostoucí komplexitou herních světů se zvyšují také nároky na chování NPC. Jednoduché skriptované reakce často nestačí k vytvoření realistického dojmu a mohou působit repetitivně nebo nepřirozeně. Návrh komplexního chování NPC proto představuje významný problém, který je nutné řešit vhodnými metodami umělé inteligence.

Jedním z přístupů k implementaci chování NPC jsou behaviorální stromy, které umožňují rozdělit rozhodovací logiku do přehledné hierarchické struktury. Tento přístup je v současnosti běžně využíván v herním vývoji díky své modularitě a schopnosti reagovat na změny prostředí.

Tato práce se zaměřuje na využití behaviorálních stromů v prostředí Unreal Engine 5 (UE5), což je moderní herní engine poskytující nástroje pro tvorbu herní umělé inteligence. Cílem práce je analyzovat principy behaviorálních stromů a demonstrovat jejich využití při implementaci chování NPC ve vzorové hře.

Struktura práce je následující. Kapitola 2 se zabývá principy behaviorálních stromů a jejich vlastnostmi. V kapitole 3 jsou popsány nástroje Unreal Engine 5, které slouží k implementaci chování NPC. Kapitola 4 představuje návrh a architekturu vytvořeného systému NPC. V kapitole 5 je popsána samotná implementace jednotlivých komponent. Kapitola 6 se věnuje testování a evaluaci vytvořeného řešení.

Kapitola 2

Behaviorální stromy

Tato kapitola popisuje základní principy behaviorálních stromů a jejich využití při návrhu chování NPC. Nejprve je vysvětleno, k čemu behaviorální stromy slouží a jakým způsobem se vykonávají. Poté jsou popsány základní typy uzlů, ze kterých se skládají. Závěr kapitoly shrnuje hlavní výhody a omezení tohoto přístupu s ohledem na návrh chování postav v UE5.

2.1 Úloha behaviorálních stromů

Behaviorální strom je způsob zápisu rozhodovací logiky autonomního agenta. V oblasti počítačových her se tímto agentem obvykle rozumí NPC, která na základě aktuální situace vybírá vhodné chování. Může například procházet herní prostředí, reagovat na hráče, komunikovat s jinou postavou nebo provádět interakce s objekty v herním prostředí.

Základní myšlenkou behaviorálního stromu je rozdělit chování na menší části. Místo jedné rozsáhlé podmínkové struktury je rozhodování popsáno pomocí uzlů uspořádaných do stromu. Některé uzly určují pořadí vyhodnocování, jiné ověřují podmínky a další vykonávají konkrétní akce. Díky tomu je možné složitější chování sestavit z jednodušších a lépe oddělených částí.[1, s. 6]

Tento způsob návrhu je vhodný zejména pro herní umělou inteligenci, protože chování postav musí být často reaktivní. Postava se nemůže rozhodnout pouze jednou a poté pokračovat bez ohledu na změny okolí. Naopak musí průběžně reagovat na pohyb hráče, změnu stavu objektů, dostupnost cesty nebo výsledek právě prováděné akce. Behaviorální strom umožňuje takové změny zachytit opakovaným vyhodnocováním rozhodovací logiky.[1, s. 5]

V této práci jsou behaviorální stromy důležité především proto, že představují hlavní prostředek pro řízení chování NPC v UE5. Pomocí nich lze jednotlivé typy chování oddělit do samostatných větví a následně určit, za jakých podmínek se mají spouštět.

2.2 Základní princip vykonávání

Behaviorální strom se vykonává od kořenového uzlu. Z něj je v pravidelných intervalech vysílán signál označovaný jako *tick*. Tento signál postupuje stromem podle pravidel jednotlivých uzlů. Uzel je tedy vyhodnocen pouze tehdy, pokud k němu *tick* dorazí.[1, s. 6]

Každý uzel po vyhodnocení vrací svému rodiči výsledek. V běžném pojetí behaviorálních stromů se používají tři základní návratové stavy. Stav *Success* znamená, že uzel byl úspěšně dokončen. Stav *Failure* vyjadřuje, že uzel nemohl svůj úkol splnit. Stav *Running* se používá v případě, kdy akce stále probíhá a její výsledek zatím není známý.[1, s. 9, tab. 1.1]

Tyto návratové stavy tvoří základ komunikace mezi jednotlivými částmi stromu. Například akce přesunu na určité místo většinou nemůže skončit během jediného snímku hry. Dokud se postava pohybuje k cíli, uzel může vracet stav *Running*. Jakmile je cíl dosažen, vrátí *Success*. Pokud se cestu nepodaří najít, výsledek bude *Failure*.

Opakované vyhodnocování stromu umožňuje postavě reagovat na aktuální stav hry. Pokud se například během pohybu změní podmínky v okolí, může být při dalším vyhodnocení zvolena jiná větev. Tím se behaviorální strom liší od jednoduchého skriptu, který pouze provede předem danou posloupnost příkazů bez průběžného přehodnocování situace.[1, s. 9]

2.3 Struktura behaviorálního stromu

Behaviorální strom je tvořen uzly, které jsou mezi sebou propojeny vztahem rodič–potomek. Výjimkou je kořenový uzel, který má vazby pouze na své potomky. Kořenový uzel stojí na začátku vyhodnocování a jednotlivé větve stromu popisují možné části chování.[1, s. 6, kap. 1.3]

Z pohledu struktury se uzly dělí do tří kategorií: kompozitní, dekorátorové a listové. Kompozitní uzly řídí tok vykonávání a určují, v jakém pořadí a za jakých podmínek se budou potomci vyhodnocovat. Dekorátorové uzly mají právě jednoho potomka a modifikují jeho chování nebo návratový stav. Listové uzly se nacházejí na koncích větví a představují buď podmínkové ověření stavu, nebo konkrétní akci. Podrobný popis každé kategorie je uveden v sekci 2.4.

Kompozitní uzly neurčují konkrétní akci ve světě hry, ale rozhodují o tom, která část stromu bude vyhodnocena. Typicky určují pořadí potomků nebo vybírají první větev, jejíž podmínky jsou splněny. Díky nim lze ve stromu vyjádřit například prioritu jednotlivých chování.

Podmínkové listové uzly slouží k ověření určitého stavu. V herním prostředí může jít například o kontrolu, zda postava vidí hráče, zda je v dosahu objekt, nebo jestli existuje platný cíl pohybu.

Akční listové uzly představují samotné činnosti postavy. Mohou spustit pohyb, vybrat nový cíl, změnit hodnotu v *Blackboard*, přehrát animaci nebo provést interakci s objektem. V UE5 jsou pro tyto části používány především uzly typu *Task*.[2, s. 76]

2.4 Základní typy uzlů

2.4.1 Kompozitní uzly

Kompozitní uzly jsou řídicí uzly, které mají jednoho nebo více potomků. Jejich úlohou je určit, v jakém pořadí a za jakých podmínek se budou potomci vyhodnocovat. Mezi nejčastěji používané *Kompozitní* uzly patří *Sequence* a *Selector* (v literatuře též zvaný jako *Fallback*). V některých systémech se používá také uzel *Parallel*.[1, s. 6–7]

2.4.1.1 Sequence

Uzel *Sequence* slouží k postupnému vykonání několika kroků. Vyhodnocuje své potomky zleva doprava a pokračuje pouze tehdy, pokud předchozí potomek skončil úspěchem. Pokud některý potomek vrátí stav *Failure*, selže celá sekvence. Pokud potomek vrátí *Running*, znamená to, že daný krok ještě probíhá, a stejný stav vrací také celá sekvence. Uzel *Sequence* vrací návratovou hodnotu *Success* pouze v tom případě, když všichni potomci tohoto uzlu mají návratovou hodnotu *Success*. [1, s. 6]

Tento uzel je vhodný pro chování, u kterého musí být splněno více kroků v přesném pořadí. Příkladem může být interakce NPC s předmětem. Postava nejprve ověří, zda je předmět dostupný, poté se k němu přesune a nakonec provede samotnou akci. Pokud už první podmínka není splněna, nemá smysl pokračovat v dalších krocích.

2.4.1.2 Selector

Uzel *Selector* slouží k výběru první vhodné možnosti. Stejně jako *Sequence* prochází své potomky zleva doprava, ale používá opačnou logiku. Jakmile některý potomek uspěje nebo stále probíhá, *Selector* se zastaví a vrátí odpovídající stav. Selže pouze tehdy, když selžou všichni jeho potomci. [1, s. 7]

Selector se často používá pro vyjádření priorit. První větev může například popisovat reakci na hráče, druhá komunikaci s jinou postavou a třetí běžné pohybování po scéně. Strom se nejdříve pokusí vykonat chování s nejvyšší prioritou. Pokud pro něj nejsou splněny podmínky, pokračuje k další možnosti.

2.4.1.3 Parallel

Uzel *Parallel* umožňuje vyhodnocovat více potomků současně. Používá se v případech, kdy má agent provádět více činností najednou nebo kdy má během jedné akce průběžně sledovat další podmínku. Chování uzlu je řízeno dvěma parametry: N označuje celkový počet přímých potomků a M je konfigurovatelný práh úspěchu ($1 \leq M \leq N$). Uzel vrátí stav *Success*, pokud alespoň M potomků uspěje, a stav *Failure*, pokud více než $N - M$ potomků selže dříve, než je práh M dosažen.[1, s. 7]

V UE5 existuje uzel *Simple Parallel*. Ten umožňuje spustit hlavní úlohu a vedle ní vyhodnocovat další větev stromu.[2, s. 76] Prakticky může jít například o situaci, kdy se postava přesouvá k cíli a zároveň kontroluje, zda se nezměnil stav okolí.

2.4.2 Dekorátorové uzly

Dekorátorové uzly představují speciální typ řídicích uzlů, které mají právě jednoho potomka a slouží k úpravě jeho chování nebo návratového stavu.[1, s. 8–9] Na rozdíl od ostatních řídicích uzlů, které určují tok vykonávání mezi více potomky, dekorátory modifikují výsledek nebo podmínky vykonání jediné podřízené větve.

V obecné teorii behaviorálních stromů mohou dekorátory například měnit návratové hodnoty uzlů (např. převrácení výsledku *Success* na *Failure*) nebo ovlivňovat způsob vykonávání, například omezením počtu opakování určité akce.[1, s. 9]

V prostředí UE5 mají dekorátory poněkud odlišnou vizuální strukturu. Jsou obvykle připojeny ke konkrétní větvi stromu a slouží především jako podmínky určující, zda může být daná větev vykonána.[2, s. 93] Nemají tedy vlastní samostatný vizuální uzel, jako tomu je u ostatních uzlů. Typickým příkladem je kontrola hodnoty uložené v *Blackboard*. Pokud je například dostupná reference na hráče, může se vykonat větev popisující reakci na hráče. V opačném případě je tato větev přeskočena a strom pokračuje jinou alternativou.

2.4.3 Listové uzly

Listové uzly se nacházejí na koncích větví behaviorálního stromu a nemají žádné potomky. Jejich úkolem je buď ověřit určitý stav, nebo provést konkrétní akci.[1, s. 6]

Podmínkové uzly kontrolují stav prostředí nebo interních dat postavy. Mohou například ověřit, zda je hráč v dosahu, zda je dostupný objekt pro interakci nebo zda postava již dokončila předchozí úkol. Výsledkem takového uzlu je zpravidla stav *Success* nebo *Failure*.[1, s. 8] Stav *Running* se u podmínkového uzlu nevyskytuje kvůli rozhodovací podstatě tohoto uzlu.

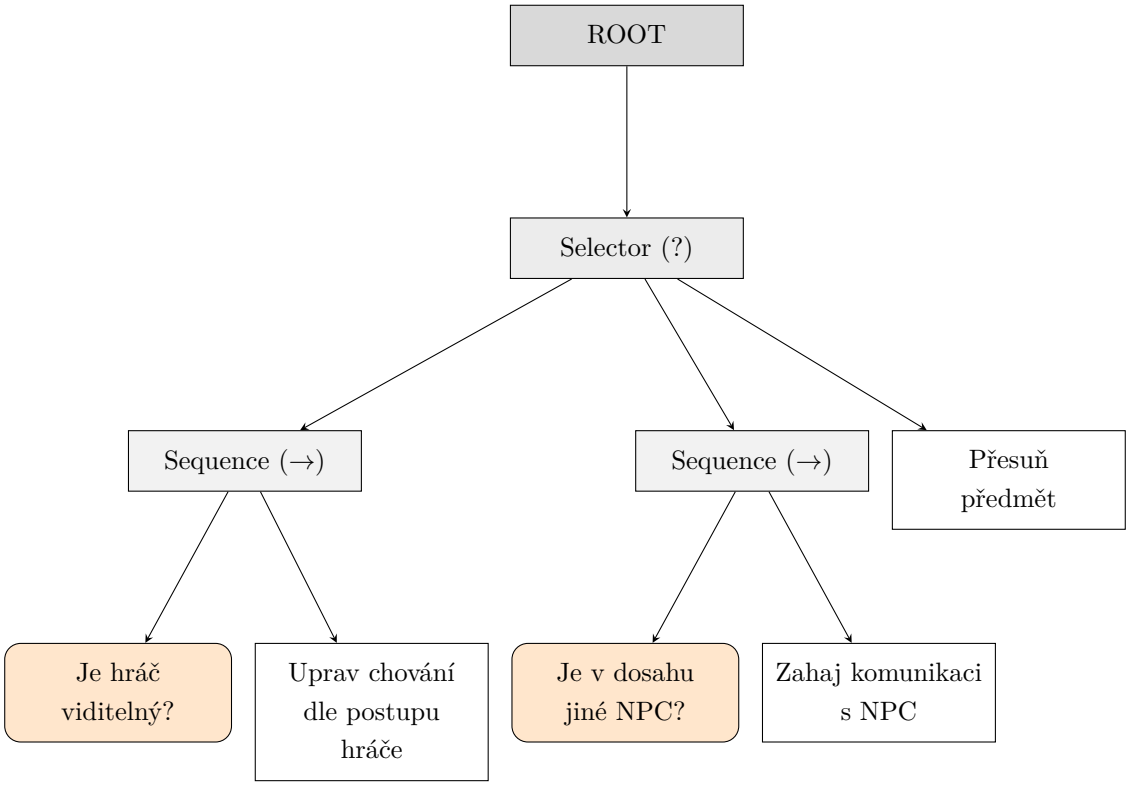
Akční uzly vykonávají konkrétní činnost. V UE5 se jedná zejména o uzly typu *Task*. Ty mohou například spustit přesun pomocí navigačního systému, vybrat náhodný bod ve scéně, změnit hodnotu v *Blackboard* nebo zavolat funkci implementovanou v *Blueprint* či C++ třídě. Právě akční uzly propojují rozhodovací logiku behaviorálního stromu se skutečným chováním postavy ve hře.

Přehled návratových stavů všech popsaných typů uzlů je uveden v tabulce 2.1.

Typ uzlu	Symbol	Success	Failure	Running
Selector	$\boxed{?}$	Pokud alespoň jeden potomek uspěje	Pokud všichni potomci selžou	Pokud alespoň jeden potomek vrátí Running
Sequence	$\boxed{\rightarrow}$	Pokud všichni potomci uspějí	Pokud alespoň jeden potomek selže	Pokud alespoň jeden potomek vrátí Running
Parallel	$\boxed{\rightrightarrows}$	Pokud $\geq M$ potomků uspěje	Pokud $> N-M$ potomků selže	V ostatních případech
Action	$\boxed{\text{text}}$	Po dokončení	Pokud nelze dokončit	Během provádění
Condition	(text)	Pokud platí (true)	Pokud neplatí (false)	Nikdy
Decorator	$\diamond$	Vlastní definice	Vlastní definice	Vlastní definice

Tabulka 2.1: Typy uzlů v behaviorálním stromě. Přeloženo podle [1, s. 9, tab. 1.1].

Výše uvedená tabulka popisuje, jak fungují návratové hodnoty jednotlivých typů uzlů, ale neméně důležité je jejich vzájemné uspořádání ve stromě. Následující diagram ukazuje, jak lze popsané uzly kombinovat na konkrétním příkladu.



Obrázek 2.1: Zjednodušená struktura behaviorálního stromu pro NPC.

Obrázek 2.1 znázorňuje, jak lze tři navrhovaná chování přirozeně uspořádat do jediného stromu. Uzel *Selector* prochází větve zleva doprava a zkouší vždy chování s nejvyšší prioritou. Nejprve se pokusí o adaptaci vůči hráči. Podmínka viditelnosti hráče musí být splněna, aby se *Sequence*

vykonala. Pokud hráč není viditelný, strom přejde na komunikaci s jiným NPC. Není-li ani to možné, vykoná se výchozí chování, kterým je přesun předmětu. Tato větev nemá podmínku, takže k ní Selector vždy dojde. Uzel může skončit úspěchem i selháním podle toho, jak skončí daná akce.

2.5 Výhody behaviorálních stromů

Jednou z hlavních výhod behaviorálních stromů je modularita. Chování lze rozdělit do menších částí, které mají jasně určenou odpovědnost. Jedna větev může řešit pohyb, jiná reakci na hráče a další například komunikaci mezi postavami. Pokud jsou tyto části navrženy obecně, lze je později použít i v jiném chování nebo u jiné postavy.[1, s. 38]

Další výhodou je přehlednost. Stromová struktura umožňuje sledovat, jakým způsobem se postava rozhoduje. To je užitečné zejména v UE5, kde lze behaviorální strom vytvářet a upravovat v grafickém editoru. Vývojář tak nemusí procházet rozsáhlý zdrojový kód, ale může pracovat s vizuální reprezentací jednotlivých větví.[1, s. 38]

Důležitá je také reaktivita. Protože se strom vyhodnocuje opakovaně, může postava reagovat na změny prostředí. Pokud se například během běžného pohybu objeví hráč, může strom při dalším vyhodnocení přejít do větve s vyšší prioritou. Díky tomu chování nepůsobí pouze jako pevně připravená posloupnost příkazů, ale může se přizpůsobovat aktuální situaci.[1, s. 38]

Behaviorální stromy také podporují postupný vývoj. Na začátku může být strom jednoduchý a obsahovat pouze několik základních větví. Později lze přidávat další podmínky, akce a reakce, aniž by bylo nutné přepsat celý systém rozhodování. To je výhodné při vývoji herní umělé inteligence, protože chování postav se často upravuje podle výsledků testování.[1, s. 38]

2.6 Omezení behaviorálních stromů

Behaviorální stromy mají také určitá omezení. U rozsáhlých stromů může postupně klesat přehlednost, zejména pokud nejsou jednotlivé větve vhodně pojmenované nebo pokud se v nich opakuje podobná logika. V takovém případě může být obtížné zjistit, proč postava zvolila právě určité chování.

Dalším problémem může být špatně navržené pořadí větví. Uzel *Selector* upřednostňuje potomky podle jejich pořadí. Pokud je důležitá větev umístěna příliš nízko, strom se k ní nemusí dostat, protože dříve uspěje jiná možnost. Chyba tedy nemusí být v samotné akci, ale ve struktuře rozhodování.

Behaviorální strom také vyžaduje průběžné vyhodnocování podmínek. U menšího počtu postav to obvykle nepředstavuje problém, ale u většího množství agentů nebo složitějších podmínek může být nutné řešit výkon. Při návrhu stromu je proto vhodné zvážit, které kontroly je nutné provádět často a které lze vyhodnocovat méně pravidelně.[1, s. 42]

2.7 Srovnání s jinými technikami implementace chování NPC

Behaviorální stromy nejsou ani zdaleka jedinou metodou řízení chování autonomních agentů. V této sekci budou představeny a porovnány behaviorální stromy se dvěma alternativními přístupy, které jsou používány v herním vývoji. Jedná se o konečné automaty (*FSM*) a systémy založenými na hodnotě užitku (*Utility-based system*).

2.7.1 Konečné automaty

Konečný automat (Finite State Machine, zkráceně FSM) je historicky nejrozšířenější přístup k implementaci chování NPC. Agent se v každém okamžiku nachází v jednom z předem definovaných stavů a přechází mezi nimi na základě podmínek, které jsou průběžně vyhodnocovány. [3, s. 318]. Pro jednoduché NPC, kterému stačí pár stavů pro jeho věrohodnou implementaci chování, FSM bohatě stačí. Asi nejlepším příkladem takových NPC jsou obecně zvířata.

Hlavním omezením FSM je škálovatelnost kvůli tomu, že s rostoucím počtem stavů roste i počet nutných přechodů mezi nimi. Každý nový stav tak může vyžadovat definování přechodů ze všech stavů ostatních. Tento jev je označován jako problém exploze stavů [1, s. 5]. Další nevýhodou je obtížná znovupoužitelnost, protože chování navržené pro jednoho agenta lze jen těžko přenést na jiného agenta s odlišnou sadou stavů.

V kontextu implementovaného systému by použití FSM vedlo k výraznému nárůstu počtu přechodů mezi stavy, zejména při kombinaci percepce, hlídkování a interakcí s objekty. Takový model by byl obtížně rozšiřitelný a méně přehledný při ladění chování.

Behaviorální stromy tento problém řeší hierarchickým uspořádáním chování do větví. Dílčí chování jsou izolována do samostatných podstromů, které lze sdílet mezi různými agenty nebo opakovaně skládat do komplexnějšího chování. Přidání nového chování nevyžaduje úpravu existujících přechodů, ale pouze přidání nové větve do stromu. Z těchto důvodů jsou behaviorální stromy považovány za přirozenou náhradu FSM v situacích, kdy počet chování přesáhne několik málo stavů [1, s. 5].

2.7.2 Systémy založené na hodnotě užitku

Systémy založené na hodnotě užitku (utility-based systems) přiřazují každé dostupné akci číselné skóre vyjadřující její vhodnost v dané situaci. Agent nejčastěji vykoná akci s nejvyšším skóre [3, s. 379]. Hodnota akce se typicky počítá jako vážená kombinace více faktorů. Například vzdálenosti od hráče, úrovně zdraví agenta nebo dostupnosti předmětů v prostředí.

Výhodou tohoto přístupu je plynulý a přirozený výběr chování. Agent nepřepíná diskrétně mezi stavy, ale průběžně reaguje na jemné změny situace. Pro simulace, kde je žádoucí, aby NPC působila organicky a ne mechanicky předvídatelně, může být tato vlastnost přínosem.

Nevýhodou je obtížnost návrhu a ladění. Správné nastavení vah a hodnotících funkcí pro každou kombinaci faktorů vyžaduje opakované testování. Výsledné chování nevyplývá z přehledné struktury, ale z numerického výpočtu, jehož průběh je těžší sledovat a opravovat.

V případě implementovaného systému by bylo nutné definovat hodnotící funkce pro kombinaci více faktorů (např. vzdálenost hráče, slyšitelnost nebo dostupnost interakcí), což by výrazně zkomplikovalo ladění chování bez zřejmého přínosu oproti explicitní hierarchii behaviorálního stromu.

Oproti tomu behaviorální strom dává návrháři explicitní kontrolu nad prioritou chování. Strukturou větví přímo určuje, co má přednost a za jakých podmínek, bez nutnosti kalibrovat numerické váhy.

2.7.3 Zhodnocení

Oproti FSM poskytují behaviorální stromy lepší škálovatelnost a přehlednost při rostoucí komplexitě chování. Ve srovnání se systémy založenými na hodnotě užitku umožňují explicitní řízení priorit a jednodušší ladění bez nutnosti definování numerických hodnotících funkcí.

Zvolený přístup se ukázal jako vhodný zejména pro implementaci hierarchického chování NPC, kde jednotlivé stavy (`Passive`, `Investigating`, `Attacking`) a jejich přechody přirozeně odpovídají struktuře behaviorálního stromu.

Kapitola 3

Unreal Engine 5 a AI systém

Tato kapitola se zabývá nástroji Unreal Engine 5, které slouží k implementaci chování NPC. Nejprve je stručně vysvětlen pojem herního enginu a představen Unreal Engine 5. Následně jsou popsány konkrétní systémy, které jsou v této práci využity pro řízení chování postav.

3.1 Herní engine a Unreal Engine 5

Herní engine je softwarový nástroj, který poskytuje základní funkcionalitu potřebnou pro vývoj her. Typicky zahrnuje subsystémy pro vykreslování grafiky, fyzikální simulaci, práci se vstupy nebo správu herní logiky. Díky tomu se vývojář nemusí zabývat implementací těchto základních částí od začátku a může se soustředit na konkrétní obsah hry.[4, s. 11–13] Herní engine tedy pomáhá zvýšit potenciální produktivitu vývojáře.

Unreal Engine 5 (UE5) je moderní herní engine vyvíjený společností Epic Games. Kromě samotného běhu hry poskytuje také nástroje pro tvorbu herního obsahu a logiky. Jednou z jeho výhod je integrace systémů pro tvorbu umělé inteligence, které umožňují navrhovat chování NPC přímo v rámci enginu bez nutnosti použití externích knihoven.[5]

V UE5 je možné implementovat herní prvky a logiku buď pomocí C++, nebo pomocí proprietárního vizuálního skriptovacího systému, který se nazývá Blueprint. V této práci jsou všechny systémy implementovány pomocí Blueprintu.

3.2 Pawn a AI Controller

V Unreal Engine 5 je objekt reprezentující postavu ve hře označován jako Pawn. Pawn určuje fyzickou přítomnost postavy ve scéně, například její polohu nebo pohyb.[6]

Každý Pawn má controller, kterým je řízen. Pokud je postava ovládána hráčem, používá se Player Controller, v případě NPC pak AI Controller.[7]

Typicky při převzetí kontroly nad Pawnem spouští behaviorální strom, který definuje jednotlivé akce NPC. Standardně platí mezi Pawn a Controller vazba 1:1, ale možná je i vícenásobná vazba (např. pro hry žánru RTS). Propojení mezi AI Controllerem a Pawnem umožňuje controlleru ovládat pohyb i další aspekty chování postavy.

3.3 Behaviorální strom v UE5

Behaviorální strom je v Unreal Engine implementován jako samostatný asset, který lze upravovat pomocí grafického editoru.[8] Princip jeho fungování odpovídá obecné teorii popsané v předchozí kapitole.

Strom se skládá z uzlů, které definují jednotlivé části chování. V UE5 se používají zejména tři typy uzlů:

- *Task* – představuje konkrétní akci (např. pohyb na cílovou pozici),
- *Service* – pravidelně aktualizuje data během vykonávání větve,
- *Decorator* – určuje podmínky, za kterých může být větev vykonána.

Behaviorální strom pracuje s daty uloženými v Blackboard a na jejich základě rozhoduje, jaká větev bude v daném okamžiku vykonána. Výhodou tohoto přístupu je přehlednost a možnost jednoduché úpravy chování bez zásahu do zdrojového kódu [2, s. 80].

3.4 Blackboard

Blackboard slouží jako sdílené datové úložiště pro behaviorální strom. Uchovává hodnoty přiřazené ke klíčům, například referenci na hráče, cílovou pozici nebo aktuální stav postavy.[8]

Samotný Blackboard neobsahuje žádnou logiku, pouze zpřístupňuje data ostatním částem systému. Tyto hodnoty mohou být nastavovány z různých míst, například z behaviorálního stromu nebo přímo z Blueprintu postavy.

3.5 Navigační systém

Pro pohyb NPC v prostředí využívá UE5 navigační systém založený na tzv. Navigation Mesh (NavMesh). NavMesh reprezentuje průchodné oblasti scény a určuje, kde se může postava pohybovat.[9]

Při vykonání akce pohybu (např. pomocí uzlu *MoveTo*) je cílová pozice předána navigačnímu systému, který vypočítá cestu po NavMesh. Samotný pohyb postavy je následně realizován komponentou pro pohyb postavy.

3.6 AI Perception

AI Perception umožňuje NPC vnímat okolní prostředí prostřednictvím definovaných smyslů, například zraku nebo sluchu [10].

Při zaznamenání podnětu (např. detekce hráče) dojde k vyvolání události, na jejímž základě se aktualizují hodnoty v Blackboardu. Behaviorální strom poté při dalším vyhodnocení reaguje na novou situaci a zvolí odpovídající chování.

Kapitola 4

Návrh a architektura NPC systému

Cílem této kapitoly je ukázat návrh architektury NPC v prostředí Unreal Engine 5, který využívá behaviorální stromy pro jeho řízení. Systém byl navržen tak, aby NPC dokázala reagovat na hráče, vnímat objekty v prostředí a v omezené míře spolupracovat s jinými NPC postavami.

V této práci bylo usilováno o dodržení oddělení odpovědností mezi jednotlivými funkčními částmi hry. Každá část systému má vymezenou roli: postava NPC zajišťuje fyzickou reprezentaci a provádění akcí, AI Controller zpracovává podněty z okolí a aktualizuje sdílenou paměť, behaviorální strom na základě těchto dat rozhoduje o aktuálním chování a Blackboard slouží jako datové úložiště propojující všechny části.

4.1 Přehled systému a jeho komponent

V projektu jsou vytvořeny dvě konkrétní NPC postavy, `BP_Grandpa` a `BP_Granny`, které sdílejí společnou rodičovskou třídu `BP_NPC`. Tato bázová třída obsahuje veškerou herní logiku sdílenou všemi NPC: systém animací, správu zbraní, detekci interaktivních objektů v okolí a propojení s globálním stavem levelu. Konkrétní odvozené třídy slouží výhradně k odlišení vizuální podoby postav – jiný skeletální mesh a animační blueprint. Přidání dalšího NPC do hry tak nevyžaduje duplikaci AI logiky.

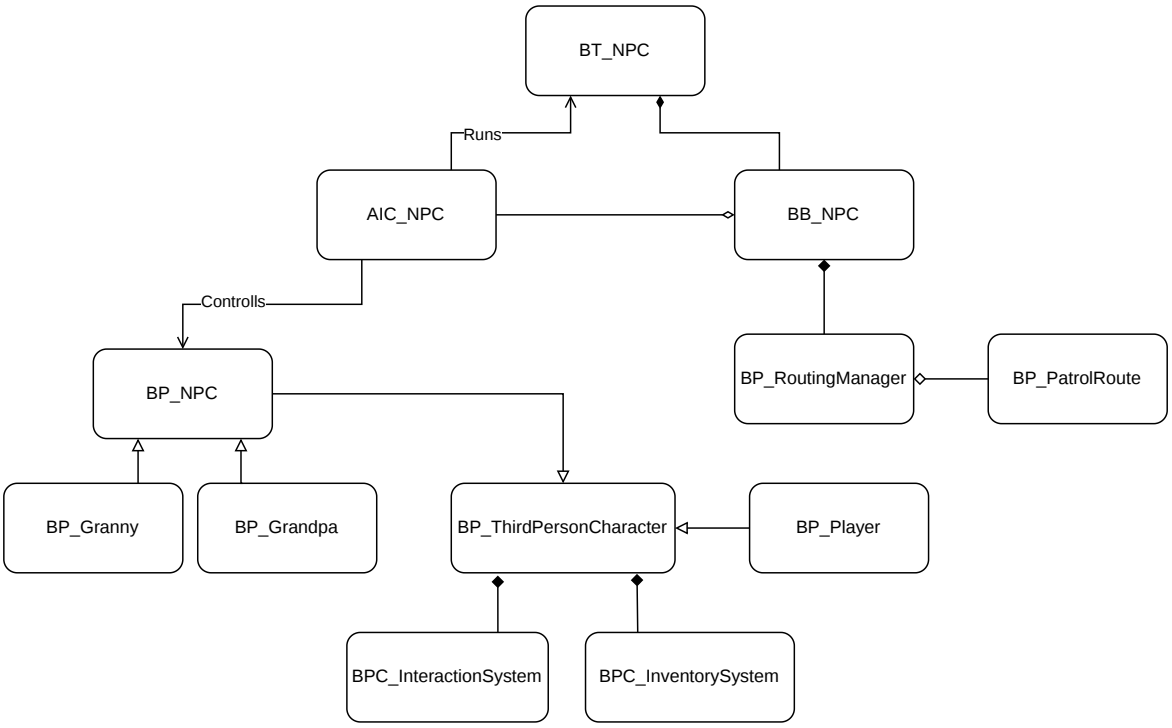
AI logika je sdílena i na vyšší úrovni: obě postavy jsou řízeny stejným AI Controllerem `IAC_NPC` a stejným behaviorálním stromem `BT_NPC`. Každá instance NPC má přitom vlastní instanci Blackboardu `BB_NPC`, takže jejich rozhodování je vzájemně nezávislé i přesto, že vychází ze stejné stromové definice.

Základní komponenty systému jsou:

- `BP_NPC` – bázová třída NPC,
- `BP_Grandpa`, `BP_Granny` – konkrétní NPC postavy,
- `IAC_NPC` – AI Controller,

- `BT_NPC` – behaviorální strom,
- `BB_NPC` – Blackboard,
- `BP_RoutingManager` – správce hlídkových tras,
- `BP_PatrolRoute` – reprezentace jednotlivé hlídkové trasy.
- `BPC_InventorySystem` – Systém inventáře, který využívají jak NPC, tak hráč
- `BPC_InteractionSystem` – Systém interakcí, který využívají jak NPC, tak hráč

Vzájemné závislosti mezi těmito komponentami jsou znázorněny na obrázku 4.1.



Obrázek 4.1: Přehled komponent systému NPC a jejich vzájemných vazeb

4.2 Rozdělení odpovědností

Návrh striktně odděluje rozhodování od vykonávání akcí. Toto oddělení je klíčovým principem využití behaviorálních stromů: strom rozhoduje, co má postava dělat, ale samotné provedení deleguje na třídu NPC.[1, s. 178]

Třída `BP_NPC` implementuje konkrétní akce – pohyb, útok, manipulaci se zbraní – a neobsahuje žádnou rozhodovací logiku. Reaguje výhradně na příkazy vydané behaviorálním stromem prostřednictvím delegátů a custom eventů. Díky tomu lze rozhodovací logiku měnit nezávisle na fyzickém chování postavy.

AI Controller `IAC_NPC` plní roli prostředníka mezi herním světem a behaviorálním stromem. Zpracovává podněty ze systému percepce, interpretuje je a přepisuje do odpovídajících klíčů Blackboardu. Controller sám nerozhoduje o chování NPC – jeho úlohou je udržovat Blackboard aktuální tak, aby behaviorální strom mohl správně reagovat.

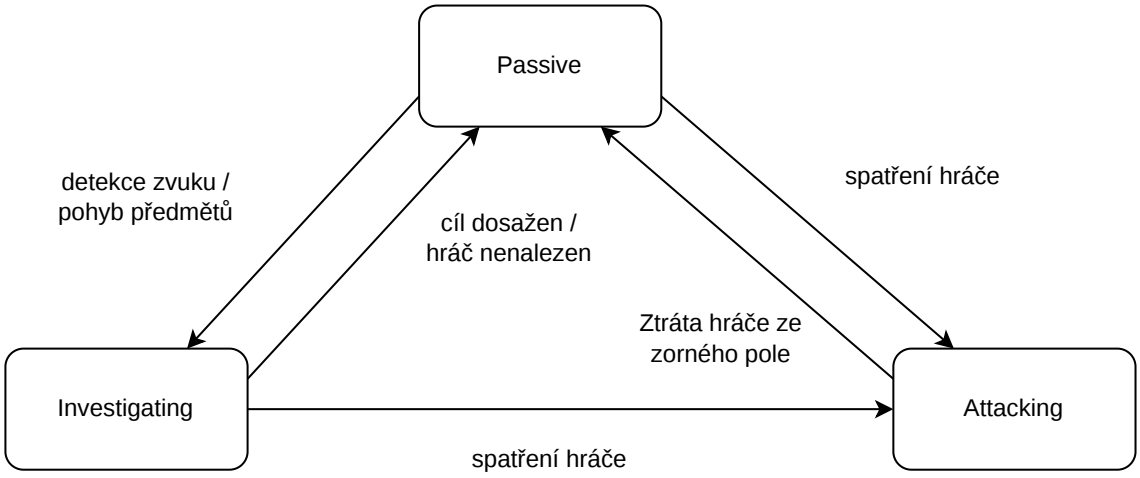
Behaviorální strom `BT_NPC` je výhradním místem rozhodovací logiky. Na základě hodnot v Blackboardu vybírá odpovídající větev a prostřednictvím Tasků vydává příkazy k provedení konkrétních akcí.

4.3 Stavový model NPC

Chování NPC je strukturováno jako třístavový model. Každý stav odpovídá jedné hlavní větvi behaviorálního stromu a určuje, která část stromu bude vyhodnocována:

- **Passive** – NPC vykonává hlídkovou rutinu, pohybuje se po přidělené trase a příležitostně interaguje s prostředím nebo jinou NPC postavou,
- **Investigating** – NPC prošetřuje podezřelý podnět, tedy detekovaný zvuk nebo pohyb sledovaného předmětu, na konkrétní souřadnici ve scéně,
- **Attacking** – NPC reaguje na viděného hráče, vytasí zbraň a zahájí útok.

Přechody mezi stavy jsou výhradně v kompetenci AI Controlleru a jsou vyvolány událostmi ze systému percepce. Spatření hráče způsobí přechod do stavu `Attacking`, ztráta hráče ze zorného pole návrat do stavu `Passive` a detekce zvuku přechod do stavu `Investigating`. Aktuální stav je uložen v Blackboardu pod klíčem `State` jako hodnota výčtového typu `E_AIState`. Schéma přechodů mezi stavy je znázorněno na obrázku 4.2.



Obrázek 4.2: Přehled hlavních stavů NPC a přechodů mezi nimi

4.4 Architektura behaviorálního stromu

Behaviorální strom `BT_NPC` je sdílen oběma NPC postavami. Každá postava spouští svou vlastní instanci stromu s vlastním Blackboardem, takže jejich rozhodování je nezávislé i při použití identické stromové struktury. Strom je spuštěn AI Controllerem při převzetí kontroly nad postavou, tedy při události `OnPossess`.

Na nejvyšší úrovni tvoří strom kořenový `Selector` se třemi větvemi. Každá větev je opatřena dekorátorem, který kontroluje hodnotu klíče `State` v Blackboardu. Selector prochází větve v pořadí od nejvyšší priority:

1. **Attacking** (nejvyšší priorita) – aktivní, pokud `State == Attacking`,
2. **Investigating** – aktivní, pokud `State == Investigating`,
3. **Passive** (výchozí) – aktivní, pokud `State == Passive`.

Tato struktura zajišťuje, že útočné chování má vždy přednost před vyšetřováním a hlídkováním, čímž je implementována přirozená hierarchie priorit. Celý behaviorální strom je příliš obsáhlý na to, aby jej bylo možné zobrazit zde, a proto jej lze nalézt v příloze této práce 1.

4.4.1 Větev Passive

Pasivní větev je nejkomplexnější částí stromu a implementuje hlídkovou rutinu NPC. Obsahuje jak služby (services), které běží periodicky na pozadí, tak úlohy (tasks) vykonávané sekvenčně:

- `BTS_EnsurePatrolRoute` (Service) – periodicky kontroluje, zda má NPC přiřazenou hlídkovou trasu; pokud ne, vyžádá ji od správce tras `BP_RoutingManager`,
- `BTT_MakeRouteDecision` (Task) – rozhoduje na základě aktuálního postupu po trase a náhodné hodnoty, zda NPC změní trasu,
- `BTT_MoveAlongPatrolRoute` (Task) – přesouvá NPC na další bod aktuální spline trasy,
- `BTT_MakeInteractionDecision` (Task) – rozhoduje, zda NPC interaguje s viděným objektem nebo s jinou NPC postavou; rozhodnutí závisí na typu interakce objektu a náhodné hodnotě `InteractionChance`,
- `BTT_GoToOtherNPC` (Task) – naviguje NPC ke druhé NPC postavě,
- `BTT_InteractWithOtherNPC` (Task) – provede výměnu informací o pozicích předmětů mezi oběma NPC,

4.4.2 Větev Attacking

Útočná větev implementuje reakci na spatřeného hráče. Po přechodu do stavu `Attacking` větev nejprve zkontroluje, zda je zbraň vytasena, a pokud ne, spustí animaci vytasení. Poté nastaví AI focus na cíl, přepne NPC do běhu a zahájí útočnou animaci:

- `BTDecorator_IsWieldWeapon` (Decorator) – podmínkově spustí vytasení zbraně pouze pokud ještě není vytasena,
- `BTT_EquipWeapon` (Task) – přehraje animaci vytasení zbraně,
- `BTT_FocusTarget` (Task) – nastaví rotační focus AI na cíl útoku uložený v Blackboardu pod klíčem `AttackTarget`,
- `BTT_SetMovementSpeed` (Task) – přepne NPC do běhu nastavením maximální rychlosti pohybu,
- `BTT_Attack` (Task) – spustí útočnou animaci; dokončení animace je signalizováno zpět do Tasku prostřednictvím delegátu.

4.4.3 Větev Investigating

Vyšetřovací větev je nejjednodušší ze všech tří. NPC naviguje na souřadnici uloženou v Blackboardu pod klíčem `PointOfInterest`. Po dosažení cílové pozice, nebo pokud cíle nelze dosáhnout, stav automaticky přejde zpět do `Passive` a NPC pokračuje v hlídkování. Tato větev se aktivuje při detekci zvuku nebo při zjištění pohybu sledovaného předmětu.

4.5 Tok dat v systému

Tok informací v systému lze popsat jako sled kroků:

`AI Perception` → `AI Controller` → `Blackboard` → `Behavior Tree` → akce NPC

AI Controller přijímá podněty ze systému percepce a přepisuje je do Blackboardu: aktualizuje stav NPC, ukládá pozici podezřelého místa nebo referenci na viděný objekt. Behaviorální strom při každém vyhodnocení přečte aktuální hodnoty z Blackboardu, vybere odpovídající větev a prostřednictvím Tasků vydá příkaz ke konkrétní akci. Samotnou akci pak vykoná třída `BP_NPC`.

4.6 Správa hlídkových tras

Hlídkové trasy jsou reprezentovány třídou `BP_PatrolRoute` jako spline-based křivky rozmístěné ve scéně. Pohyb po trase je obousměrný: NPC se pohybuje po bodech spline v jednom směru a po dosažení krajního bodu se otočí a vrátí se zpět po téže trase. Pro uzavřené smyčky je index bodu zabalován modulem délky trasy.

Přidělování tras jednotlivým NPC zajišťuje třída `BP_RoutingManager`. Tento správce řeší problém kolize tras: zamezuje situaci, kdy by dvě NPC hlídkovaly na stejné trase a pohybovaly se synchronně. Při vyžádání nové trasy systém iteruje dostupné trasy v náhodně zamíchaném pořadí a přidělí první volnou. Obsazenost tras je evidována v interní mapě.

Součástí návrhu jsou tři sady tras odpovídající různým herním fázím (`Stage 1`, `Stage 2`, `Stage 3`). Příslušnost tras k dané fázi je určena tagem přiřazeným přímo instancím `BP_PatrolRoute` ve scéně. Změna herní fáze tak přiřadí NPC jiné trasy bez jakékoliv modifikace behaviorálního stromu.

Variabilita chování je dále zajištěna pravděpodobnostním prvkem: každé NPC má při startu inicializovanou náhodnou hodnotu `RouteChangeChance` v rozsahu 0–1, která určuje pravděpodobnost dobrovolné změny trasy po dosažení určitého úseku. Práh pro toto rozhodnutí je po každé změně rovněž randomizován v rozsahu 0,25–0,75, takže NPC mění trasy nepravidelně a jejich pohyb nepůsobí mechanicky.

4.7 Reprezentace znalostí NPC

Chování NPC v této práci vyžaduje, aby postavy dokázaly pracovat s informacemi o objektech v herním prostředí. Nejedná se pouze o okamžitou reakci na podněty, ale také o schopnost zohlednit stav herního světa při rozhodování.

Znalosti NPC jsou reprezentovány kombinací globálního sdíleného stavu a lokálních dat jednotlivých postav.

4.7.1 Globální stav objektů

Informace o objektech ve scéně jsou spravovány centrálně pomocí třídy `BP_GameStateBase`. Tato třída obsahuje registr sledovaných objektů, ve kterém jsou uloženy jejich referenční informace.

Každý objekt typu `BP_BaseItem` se při inicializaci zaregistruje do tohoto registru a uloží svou výchozí pozici. Tato pozice představuje očekávaný stav objektu ve scéně.

Díky tomuto přístupu mají všechny NPC přístup ke konzistentní reprezentaci světa, aniž by bylo nutné udržovat redundantní kopie těchto dat pro každou postavu zvlášť.

4.7.2 Lokální znalosti NPC

Každá NPC si zároveň udržuje vlastní seznam známých objektů, který je uložen v její instanci. Tento seznam obsahuje reference na objekty, se kterými postava přišla do kontaktu nebo o nich získala informaci.

Lokální seznam slouží k:

- uchování informací relevantních pro konkrétní NPC,
- vyhodnocování interakcí s objekty,
- rozhodování v rámci behaviorálního stromu.

Tato data nejsou sdílena přímo mezi NPC, ale každá postava je interpretuje samostatně na základě globálního stavu a vlastních zkušeností.

4.7.3 Inicializace znalostí

Při spuštění hry dojde k inicializaci registru objektů a jejich základních vlastností. NPC následně pracují s těmito daty během svého běžného chování, zejména v rámci pasivní větve behaviorálního stromu.

4.7.4 Detekce změn v prostředí

Změna stavu objektu je určena porovnáním jeho aktuální pozice s referenční hodnotou uloženou v globálním registru.

Pokud se tyto hodnoty liší, je objekt považován za přesunutý nebo chybějící. Tato informace je následně využita při rozhodování NPC, například pro přechod do stavu *Investigating*.

Tímto způsobem je možné detekovat změny ve světě bez nutnosti složitějších mechanismů sledování historie.

4.8 Komunikace mezi NPC

Komunikace mezi NPC je realizována prostřednictvím Tasku `BTT_InteractWithOtherNPC`, který se spouští v rámci pasivní větve behaviorálního stromu, když se dvě NPC dostatečně přiblíží. Mechanismus výměny informací probíhá ve třech krocích.

Nejprve aktivní NPC zjistí z globálního herního registru, tedy z `BP_GameStateBase`, jaká je domácí neboli referenční pozice sledovaného předmětu – tedy kde by měl předmět být. Poté přečte ze záznamu paměti druhé NPC, kde tento předmět druhá NPC naposledy viděla. Nakonec obě hodnoty porovná.

Pokud jsou pozice shodné, předmět nebyl přemístěn a NPC pokračují v hlídce. Pokud se pozice liší, byl předmět přemístěn hráčem. V takovém případě jsou u obou NPC nastaveny Blackboard klíče `LostItem` a `SearchShed` na hodnotu `true`, čímž jsou obě postavy instruovány k prohledání určitého místa. Zápis do Blackboardu druhého NPC probíhá přímým přístupem přes jeho AI Controller, bez použití centrálního koordinátora.

Jedná se o decentralizovaný mechanismus spolupráce, což znamená, že každé NPC si informace aktivně zjišťuje při každém setkání s partnerem a v případě nesouladu koordinovaně zahájí vyšetřování. Tím je naplněn požadavek na implementaci spolupráce mezi NPC postavami.

4.9 Reakce na hráče a prostředí

Reakce NPC na hráče je řízena systémem AI Perception implementovaným v AI Controlleru. Systém percepce pracuje se dvěma smysly: zrakem (`AISense_Sight`) a sluchem (`AISense_Hearing`).

Při zpracování vizuálních podnětů rozlišuje Controller tři scénáře. Pokud NPC spatří hráče, okamžitě přejde do stavu `Attacking` – behaviorální strom při dalším vyhodnocení aktivuje útočnou větev s nejvyšší prioritou. Pokud hráč opustí zorné pole, NPC se vrátí do stavu `Passive`. Pokud NPC zaznamená interaktivní objekt ve scéně, uloží referenci na něj do Blackboardu pod klíč `SeenInteractible`, který pasivní větev stromu využije při rozhodování o interakci.

Zvukové podněty zpracovává Controller odděleně: pokud NPC uslyší zvuk a aktuálně není ve stavu `Attacking`, přejde do stavu `Investigating` s cílovou souřadnicí zdroje zvuku. Pokud NPC již útočí, zvuk je ignorován – útok má vyšší prioritu a přechod do vyšetřovacího stavu by ho nežádoucně přerušil.

4.10 Návrhová rozhodnutí

Při návrhu systému byla přijata následující klíčová rozhodnutí:

- **Společná bázová třída NPC** – odděluje vizuální identitu od AI logiky a umožňuje přidávání nových typů NPC bez duplikace kódu. Nová postava vyžaduje pouze nový mesh a animační blueprint.

- **Sdílený behaviorální strom s nezávislými Blackboardy** – obě NPC sdílejí jednu definici stromu, ale každá rozhoduje na základě vlastních dat. Tím je zajištěna konzistence chování při minimálním počtu assetů.
- **Oddělení rozhodování od vykonávání** – AI Controller a behaviorální strom rozhodují, třída NPC vykonává. Toto oddělení odpovídá doporučenému architektonickému vzoru pro využití behaviorálních stromů v Unreal Engine 5.
- **Blackboard jako centrální sdílená paměť** – propojuje Controller, strom a Tasky bez přímých závislostí mezi nimi; každá část systému čte a zapisuje pouze do Blackboardu.
- **Pravděpodobnostní inicializace chování** – náhodná inicializace hodnot `RouteChangeChance` a `InteractionChance` při startu každého NPC zajišťuje, že i při sdíleném stromu se obě postavy chovají mírně odlišně a jejich pohyb nepůsobí synchronně.
- **Správce tras bez kolizí** – `BP_RoutingManager` zamezuje obsazení stejné trasy dvěma NPC současně, čímž zvyšuje přirozenost hlídkování a zabraňuje vizuálně rušivé synchronizaci pohybu.

Kapitola 5

Implementace systému NPC

Implementace byla rozdělena do několika navazujících částí. Nejprve je popsána realizace základní třídy NPC a jejích odvozených variant, poté implementace AI Controlleru, Blackboardu a behaviorálního stromu. Následující části se věnují konkrétním Taskům a Services, správě hlídkových tras, zpracování percepčních podnětů a komunikaci mezi NPC. Závěr kapitoly shrnuje vybraná implementační rozhodnutí a problémy, které bylo nutné při vývoji řešit.

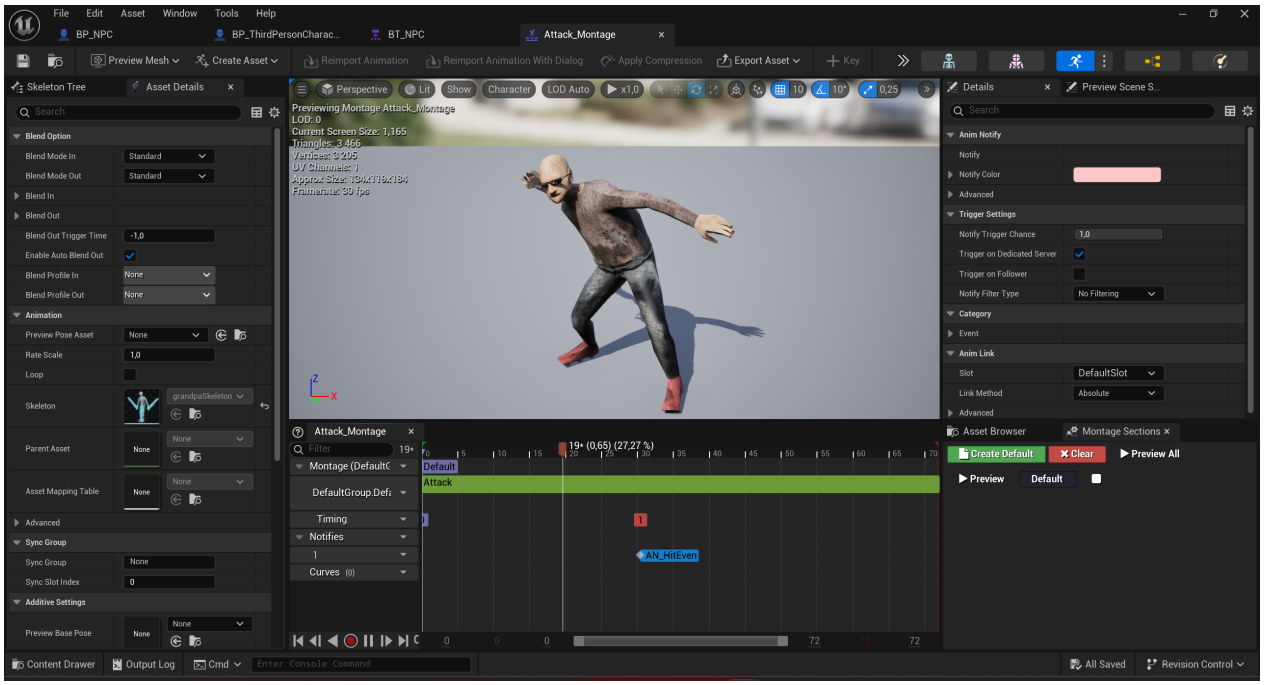
5.1 Implementace BP_NPC

Třída `BP_NPC` je odvozena od třídy `BP_ThirdPersonCharacter` a představuje základní implementaci všech nehráčských postav ve hře. Jejím hlavním účelem je realizace konkrétních akcí ve světě, zatímco rozhodovací logika je delegována na AI Controller a behaviorální strom. Třída tak slouží především jako vykonavatel příkazů generovaných vyššími vrstvami systému.

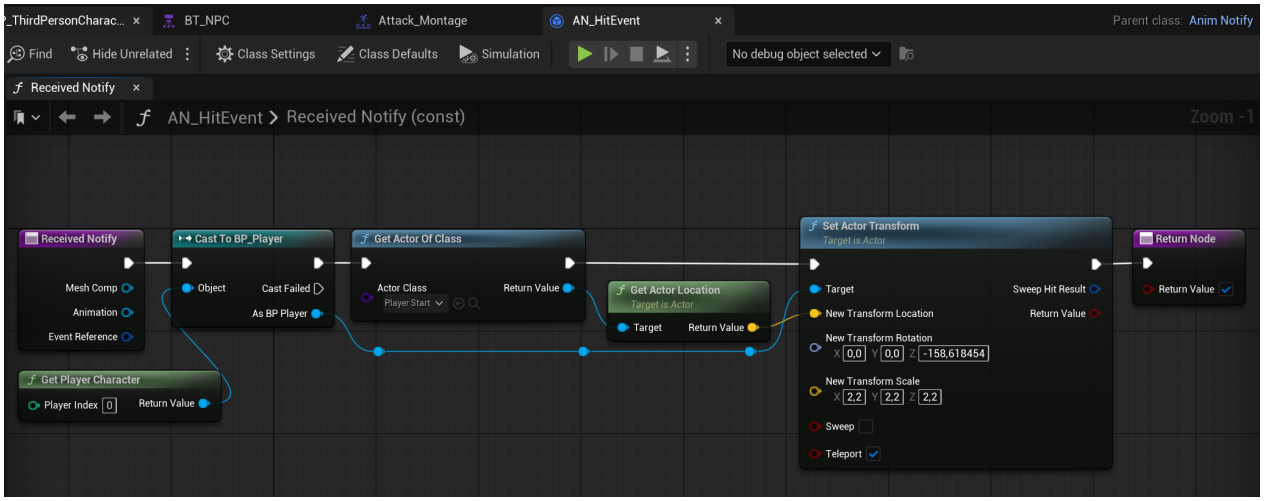
V rámci třídy jsou implementovány základní mechaniky pohybu, animací, útoku a interakcí s okolím. Aktualizace stavových proměnných souvisejících s pohybem probíhá periodicky v rámci události `Tick`, kde je z komponenty `CharacterMovement` získána aktuální rychlost postavy. Její velikost je následně ukládána do proměnné `Speed`, která slouží jako vstup pro animační systém. Současně je pomocí funkce `IsFalling` určováno, zda se postava nachází ve vzduchu, a výsledek je ukládán do proměnné `IsJumping`. Tyto proměnné umožňují animačnímu blueprintu plynule přepínat mezi jednotlivými animačními stavy.

Útočné chování je realizováno prostřednictvím vlastní události `Attack`, která je volána z behaviorálního stromu, k čemuž slouží *task* `BTT_Attack`. Po jejím spuštění je nastavena proměnná `IsAttacking` a přehrána odpovídající animační montáž pomocí uzlu `Play Montage`. Po dokončení nebo přerušení animace je proměnná `IsAttacking` opět resetována. Za povšimnutí stojí systém notifikací. V rámci animace (respektivě animační montáže) může vývojář vložit události na konkrétní snímky animace, čímž je mu dovoleno velmi flexibilně ovlivňovat dění v herním světě v průběhu animace. Pro využití systému notifikací stačí vložit notifikaci na časové ose animace a zvolit buď

jeden z předpřipravených událostí (např. *Play Sound*), nebo vlastní událost, jak jde vidět na obrázku 5.1. Obrázek 5.2 poté ukazuje, jak taková událost může být implementována. Tato konkrétní událost při zásahu hráče teleportuje hráče zpátky na `PlayerStart` (začátek levelu).



Obrázek 5.1: Umístění události `AN_HitEvent` v animační montáži



Obrázek 5.2: Implementace události `AN_HitEvent` v blueprintu

Manipulace se zbraní je řešena pomocí dvojice událostí `EquipWeapon` a `UnequipWeapon`. Událost `EquipWeapon` přehrává animační montáž tasení zbraně a následně vytváří instanci zbraně pomocí `SpawnActor`. Vytvořený objekt je připojen ke kostře postavy na konkrétní socket. Reference na

vytvořenou zbraň je uložena do proměnné `WeaponActor` a příznak `IsWieldingWeapon` indikuje, že postava aktuálně drží zbraň. Opačný proces je implementován v události `UnequipWeapon`, kde je zbraň odpojena a odstraněna.

Interakce s okolními objekty je realizována prostřednictvím kolizního systému. Při vstupu do kolize s objektem implementujícím rozhraní `BPI_Interactible` je tato skutečnost detekována v události `OnComponentBeginOverlap`. Následně je získán AI Controller dané postavy a prostřednictvím jeho rozhraní je aktualizována odpovídající hodnota v Blackboardu, čímž je behaviorálnímu stromu signalizována možnost interakce. Při ukončení kolize (`OnComponentEndOverlap`) je tato informace opět odstraněna.

Třída dále obsahuje pomocnou funkci `SetMovementSpeed`, která na základě hodnoty výčtového typu nastavuje maximální rychlost pohybu postavy.

Konkrétní postavy `BP_Grandpa` a `BP_Granny` dědí z třídy `BP_NPC` a upravují především vizuální reprezentaci a animační parametry. Sdílená logika chování zůstává v třídě rodiče, čímž je minimalizována duplicita a zjednodušena případná rozšiřitelnost systému.

5.2 Implementace AI Controlleru (IAC_NPC)

Třída `IAC_NPC`, odvozená od `AIController`, představuje hlavní řídicí komponentu NPC. Jejím úkolem je zprostředkovat komunikaci mezi percepčním systémem, behaviorálním stromem a herním světem, a zároveň udržovat aktuální stav postavy prostřednictvím *blackboardu*.

5.2.1 Inicializace a napojení na behaviorální strom

Po převzetí řízení nad postavou (událost `OnPossess`) dochází k inicializaci celého systému. Nejprve jsou vyhledány všechny instance tříd reprezentujících hlídkové trasy a uloženy do interní kolekce. Tento krok umožňuje dynamicky reagovat na změny ve scéně bez nutnosti pevného navázání tras v editoru.

Následně je spuštěn behaviorální strom `BT_NPC`, přičemž před jeho spuštěním jsou do *blackboardu* zapsány výchozí hodnoty. Jedná se zejména o:

- referenci na řízenou postavu (`SelfActor`),
- pravděpodobnost změny hlídkové trasy,
- pravděpodobnost interakce s objektem,
- počáteční hlídková trasa.

Inicializační fáze tak zajišťuje, že behaviorální strom pracuje nad konzistentní sadou dat a může okamžitě začít rozhodovací proces.

5.2.2 Zpracování percepce

Pro vytvoření dojmu smyslového vnímání u NPC se využívá komponenta `AIPerception`, která agreguje informace o okolních aktérech a vyvolává událost `OnTargetPerceptionUpdated` při každé změně percepčního stavu.

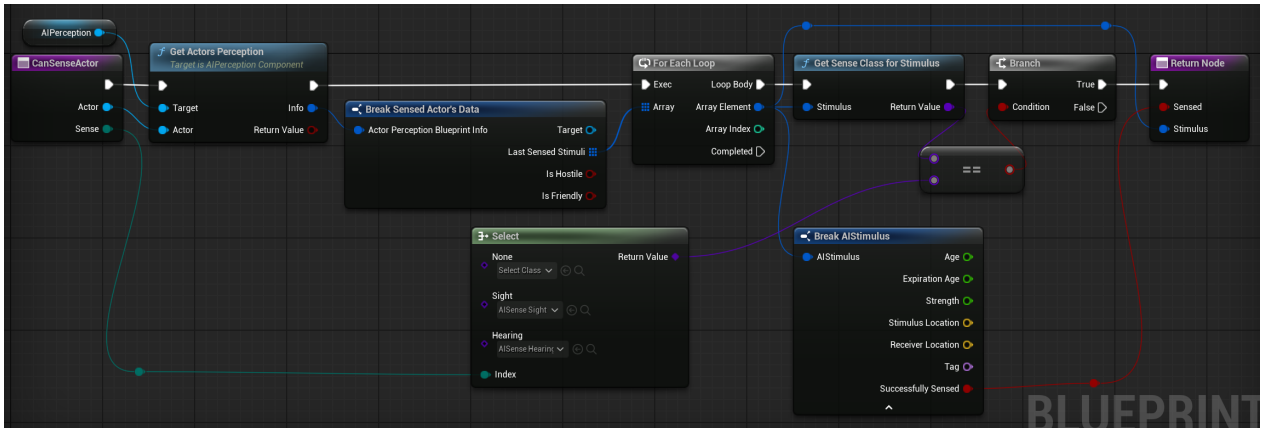
Zpracování této události probíhá tak, že pro každý aktualizovaný aktér jsou získána percepční data, jednotlivé podněty jsou analyzovány podle typu smyslu a na základě výsledku jsou následně aktualizovány klíče v *blackboardu*.

Pro rozlišení typu podnětu je využita pomocná metoda `CanSenseActor`, jejíž implementace je zobrazena na obrázku 5.3. Metoda nejprve získá percepční data pro daného aktéra pomocí uzlu `Get Actors Perception`. Tato data jsou následně rozložena na jednotlivé stimuly, ze kterých se percepční informace skládá.

Následně je ověřeno, zda některý ze stimulů odpovídá požadovanému typu smyslu.

Pokud dojde ke shodě typů, metoda vrací informaci o úspěšném zaznamenání podnětu spolu s odpovídajícím stimulem. V opačném případě pokračuje vyhodnocování dalších stimulů, dokud není nalezen odpovídající podnět nebo není seznam vyčerpán.

Pro obě NPC je nastaven zrak (`AISense_Sight`) jako dominantní smysl. Dominance vjemu znamená, že pokud je v rámci jedné aktualizace percepce zaznamenáno více podnětů současně, je prioritně vyhodnocován právě tento smysl.



Obrázek 5.3: Implementace metody CanSenseActor

5.2.2.1 Zrakové podněty

Zrakové podněty jsou zpracovávány funkcí `HandleSensedSight`, která navazuje na funkci `CanSenseActor`. Tato funkce přijímá referenci na detekovaného aktéra a hodnotu určující, zda byl aktér aktuálně úspěšně zaznamenán. Na rozdíl od sluchového podnětu zde nestačí pouze uložit polohu stimulu, protože zrak může detekovat různé typy objektů s odlišným významem pro chování NPC.

Prvním krokem je ověření, zda je detekovaným aktérem hráč. Pokud ano a podnět je úspěšně zaznamenán, funkce zjistí aktuální stav NPC pomocí `GetCurrentState`. Nenachází-li se NPC ve stavu `Attacking`, je zavolána funkce `SetStateAsAttacking`. Ta zapíše referenci na hráče do klíče `AttackTarget` a nastaví klíč `State` na hodnotu `Attacking`. Tím je behaviorálnímu stromu předána informace, že má přejít do větve útočného chování.

Druhou částí zpracování zrakového podnětu je práce s interaktivními objekty. Funkce ověřuje, zda detekovaný aktér implementuje rozhraní `BPI_Interactible`. Pokud ano, je porovnán s hodnotou aktuálně uloženou v klíči `SeenInteractible`. V případě platného nově zaznamenaného objektu je reference na tento objekt uložena do *blackboardu*. Pokud naopak NPC daný objekt již nezaznamenává a tento objekt odpovídá aktuálně uložené hodnotě, je klíč `SeenInteractible` vymazán.

Tato logika umožňuje, aby jeden percepční mechanismus sloužil jak pro reakci na hráče, tak pro průběžné sledování objektů, se kterými může NPC později interagovat. Rozhodnutí, zda s objektem skutečně interagovat, však není provedeno přímo v AI Controlleru, ale až v příslušné části behaviorálního stromu.

5.2.2.2 Sluchové podněty

Sluchové podněty jsou zpracovávány funkcí `HandleSensedSound`. Oproti zrakovým podnětům je tato část implementace jednodušší, protože zvukový podnět je reprezentován pouze polohou, nikoliv konkrétním cílovým aktérem.

Funkce nejprve zjistí aktuální stav NPC pomocí `GetCurrentState`. Pokud se postava nenachází ve stavu `Investigating`, je zavolána funkce `SetStateAsInvestigating`. Ta nastaví klíč `State` na hodnotu `Investigating` a uloží polohu zvukového podnětu do klíče `PointOfInterest`. Behaviorální strom tak při dalším vyhodnocení přejde do větve vyšetřování a NPC se přesune k místu, odkud byl podnět zaznamenán.

Pokud se NPC nachází ve stavu `Attacking`, ke změně na vyšetřovací stav nedochází. Tím je zajištěno, že zvukový podnět nepřeruší probíhající útočné chování.

5.2.3 Správa stavů

Aktuální stav NPC je reprezentován výčtovým typem `E_AIState` a uložen v *blackboardu*.

Pro práci se stavem jsou implementovány pomocné metody:

- `GetCurrentState` – vrací aktuální stav,
- `SetStateAsPassive` – nastaví pasivní chování,
- `SetStateAsAttacking` – nastaví útočný režim,
- `SetStateAsInvestigating` – nastaví režim vyšetřování.

Každá z těchto metod provádí nejen změnu stavu, ale i odpovídající aktualizaci dalších klíčů (např. cíle útoku nebo bodu zájmu). Tím je zajištěno, že behaviorální strom vždy pracuje s konzistentními daty.

5.2.4 Interakce s objekty

Součástí implementace je i podpora interakcí s objekty ve scéně. Při detekci aktéra, který implementuje rozhraní pro interakci, je jeho reference uložena do klíče `CurrentInteractionTarget`. Pokud se jedná o nový objekt (odlišný od posledního), je tato změna propagována do *blackboardu*.

Další metody umožňují reagovat na konkrétní události:

- `NotifyItemPickUp` – zaznamená typ drženého objektu,
- `NotifyItemPlaced` – vymaže informaci o drženém objektu.

Tyto informace jsou následně využívány behaviorálním stromem pro rozhodování o dalším postupu.

5.3 Implementace stromu chování (BT_NPC)

Strom chování `BT_NPC` se větví do tří hlavních větví (`Attacking`, `Investigating`, `Passive`).

Chování NPC je dále ovlivněno stavem několika klíčů v *blackboard*, které reprezentují aktuální fázi hry. Na základě těchto hodnot dochází ke změnám v dostupných hlídkových trasách.

- `Fáze hledání` – hráč hledá klíč od venkovní boudy,
- `Fáze pátrání` – hráč odemknul venkovní boudu,
- `Fáze útěku` — hráč se snaží utéct z pozemku.

Přechody mezi jednotlivými fázemi jsou řízeny pomocí *blackboard klíčů* `SearchShed` a `IsShedOpen`, které odrážejí postup hráče ve hře.

Tato struktura přirozeně vyjadřuje prioritizaci chování: útok má přednost před pátráním a pátrání má přednost před hlídkováním. NPC zmenšují počet hlídkovacích tras, které si vybírají k hlídkování a zvyšují svůj pobyt venku, aby zamezili útěku hráče.

5.3.1 Větev Attacking

Větev `Attacking` reprezentuje chování NPC v situaci, kdy je detekován nepřítel. Je aktivována na základě hodnoty klíče `State` v *blackboardu*, který je nastaven na `Attacking` v rámci zpracování percepčních podnětů.

V této větvi NPC využívá klíč `AttackTarget`, který obsahuje referenci na aktuální cíl. Na základě této hodnoty dochází k pohybu směrem k cíli a následnému provedení útoku.

Samotný útok je realizován pomocí úlohy `BTT_Attack`, která spouští odpovídající animační montáž (viz 5.1) a řídí průběh útočné akce.

Chování v této větvi má nejvyšší prioritu, což znamená, že při detekci nepřítele dochází k okamžitému přepnutí z ostatních stavů do útočného režimu.

5.3.2 Větev Investigating

Větev `Investigating` reprezentuje chování NPC při vyhodnocení podezřelého podnětu, typicky na základě sluchového vjemu.

Aktivace této větve je řízena hodnotou klíče `State` v *blackboard*, který je nastaven na `Investigating` ve funkci `HandleSensedSound`. Současně je do klíče `PointOfInterest` uložena poloha, odkud byl podnět zaznamenán.

V rámci této větve se NPC přesouvá k uložené pozici a provádí její prohledání. Konkrétní průběh tohoto chování je řízen behaviorálním stromem na základě hodnoty klíče `PointOfInterest`.

Pokud během vyšetřování dojde k detekci nepřítele, dochází ke změně stavu na `Attacking`, což má vyšší prioritu a způsobí opuštění této větve. Pokud se naopak ukáže, že prozkoumávané místo není ničím důležité, tak dojde ke změně do stavu `Passive`.

5.3.3 Větev Passive

Větev `Passive` reprezentuje výchozí chování NPC v situaci, kdy není detekován žádný významný podnět. Slouží jako základní režim, ve kterém NPC vykonává hlídkování.

V rámci této větve je využita služba `BTS_EnsurePatrolRoute`, která zajišťuje, že má NPC přiřazenou platnou hlídkovou trasu. V případě, že žádná trasa není nastavena, služba vyžádá její přidělení prostřednictvím systému `BP_RoutingManager`.

Samotný pohyb po trase je realizován úlohou `BTT_MoveAlongPatrolRoute`, která postupně naviguje NPC mezi jednotlivými body hlídkové trasy.

Taktéž v této větvi probíhají interakce `BTT_InteractWithActor`, přemisťování předmětů `BTT_PlaceItem`, komunikace s jiným NPC `BTT_InteractWithOtherNPC` a dělání rozhodnutí o změnách hlídkových tras.

Pokud NPC nemá k dispozici žádnou trasu, setrvává na místě do doby, než je jí trasa přidělena. Tato větev tak zajišťuje kontinuální chování NPC i v případě absence dalších podnětů z okolí.

5.4 Implementace úloh a služeb

Tato část popisuje vybrané úlohy a služby behaviorálního stromu, které přímo propojují rozhodovací logiku stromu s konkrétními akcemi NPC ve světě hry. Úlohy typu *Task* slouží k provedení jednorázové nebo déle trvající akce, zatímco služby typu *Service* průběžně aktualizují data potřebná pro vykonávání dané větve stromu.

5.4.1 BTT_Attack

Úloha `BTT_Attack` realizuje spuštění útoku řízené NPC postavy. Při vykonání úlohy je nejprve získána reference na ovládanou postavu a ta je přetypována na `BP_NPC`. Následně je na této postavě zavolána událost `Attack`, která je implementována v třídě `BP_NPC` a zajišťuje přehrání útočné animační montáže.

Úloha není ukončena okamžitě po spuštění animace. Místo toho se naváže na událost `OnAttackEnd`, která je vyvolána až po dokončení útoku. Teprve poté je zavoláno `Finish Execute` s výsledkem `Success`. Tím je zajištěno, že behaviorální strom pokračuje až ve chvíli, kdy je útočná akce skutečně dokončena.

Součástí úlohy je také obsluha přerušení. Pokud je úloha předčasně ukončena, například kvůli přechodu do jiné větve stromu, je zastaven pohyb controlleru, zrušen aktuální focus a úloha je ukončena pomocí `Finish Abort`. Tento postup zabraňuje tomu, aby NPC po přerušení útoku pokračovalo v neaktuální akci.

5.4.2 BTT_MoveAlongPatrolRoute

Úloha `BTT_MoveAlongPatrolRoute` zajišťuje pohyb NPC po aktuálně přiřazené hlídkové trase. Při spuštění úloha načte z *blackboardu* hodnotu klíče `CurrentPatrolRoute` a zkontroluje se, zda je klíč platný.

Z trasy je následně získána světová pozice aktuálního bodu pomocí funkce `GetSplinePointAsWorldPosition`. Na tuto pozici je NPC navigována pomocí uzlu `AI MoveTo`. Po úspěšném dosažení cíle je na trase zavolána funkce `IncrementPatrolRoute`, která posune interní ukazatel na další bod trasy. Poté je úloha ukončena pomocí `Finish Execute` s výsledkem `Success`.

Pokud se nepodaří získat platnou trasu, přetypování selže nebo pohyb k cílové pozici není úspěšný, úloha je ukončena neúspěšně. Součástí implementace je také reakce na přerušení úlohy, při které je pohyb NPC zastaven a úloha je ukončena pomocí `Finish Abort`.

5.4.3 BTS_EnsurePatrolRoute

Služba `BTS_EnsurePatrolRoute` zajišťuje, že má NPC během pasivního chování přiřazenou platnou hlídkovou trasu. Při aktivaci služby je z *blackboardu* načten klíč `CurrentPatrolRoute`. Pokud je tato hodnota již nastavena, není nutné trasu měnit.

V případě, že NPC platnou trasu nemá, služba načte z *blackboardu* referenci na systém `BP_RoutingManager`. Po ověření platnosti této reference je řízená postava přetypována na `BP_NPC` a je zavolána funkce `ChangeRoute`. Výsledkem této funkce je nová hlídková trasa, která je následně uložena zpět do klíče `CurrentPatrolRoute`.

Tato služba odděluje samotné přidělování tras od úlohy, která po trase pouze vykonává pohyb. Díky tomu může být logika výběru trasy spravována centralizovaně v `BP_RoutingManager`, zatímco behaviorální strom pracuje pouze s aktuálně přiřazenou trasou.

5.5 Implementace hlídkových tras

5.5.1 BP_PatrolRoute

Třída `BP_PatrolRoute` reprezentuje hlídkovou trasu pomocí komponenty typu `SplineComponent`, která definuje plynulou křivku ve světovém prostoru herní úrovně. Jednotlivé hlídkové body nejsou explicitně reprezentovány samostatnými komponentami, ale odpovídají bodům spline křivky.

Pro práci s trasou třída poskytuje několik metod. Metoda `GetSplinePointAsWorldPosition` vrací světovou polohu bodu spline se zadaným indexem. Dále je implementována metoda `IncrementPatrolRoute`, která aktualizuje index aktuálního bodu na základě směru pohybu a typu trasy.

Trasa může být buď cyklická, nebo otevřená. V případě cyklické trasy je index inkrementován o modulo počtu bodů křivky. U otevřené trasy dochází při dosažení koncových bodů ke změně směru průchodu (tzv. "ping-pong" pohyb), což je realizováno přepínáním hodnoty proměnné `Direction`.

Pomocné metody `GetRouteProgress` a `GetRouteType` umožňují získat relativní postup po trase a určit, zda se jedná o cyklickou či necyklickou trasu.

5.5.2 BP_RoutingManager

Třída `BP_RoutingManager` implementuje správu hlídkových tras a jejich přidělování NPC postavám.

Při inicializaci v rámci události `BeginPlay` jsou vyhledány všechny dostupné hlídkové trasy v levelu a ty jsou uloženy do interních kolekcí. Trasy jsou dále rozděleny do skupin na základě tagů aktérů (např. `STAGE2`, `STAGE3`), což umožňuje řídit, které trasy jsou dostupné v různých fázích hry.

Obsazenost tras není reprezentována přímo v objektech tras, ale je evidována v mapě `Occupants`, která mapuje instance NPC na jim přidělené trasy.

Metoda `AcquireRoute(Requester)` vrací trasu pro daného žadatele. Pokud již má NPC trasu přidělenou, je vrácena tato existující trasa. V opačném případě dojde k náhodnému promíchání seznamu tras a následně k výběru první neobsazené trasy. Vybraná trasa je zapsána do mapy `Occupants`.

Metoda `IsRouteOccupied(Route)` ověřuje, zda je daná trasa aktuálně obsazena některou NPC postavou.

Metoda `ReleaseRoute(Requester)` odstraní záznam o přiřazení trasy danému NPC a tím ji uvolní pro další použití.

Metoda `ChangeRoute(Requester)` nejprve uvolní aktuální trasu a následně přidělí novou trasu pomocí `AcquireRoute`. Specifické varianty `ChangeRoute_Stage2` a `ChangeRoute_Stage3` vybírají trasu pouze z příslušné podmnožiny tras definované tagy.

Pokud není žádná volná trasa k dispozici, metoda vrací hodnotu `None`, přičemž NPC může o přiřazení požádat znovu v dalším průchodu stromem.

5.6 Sdílení informací o předmětech

Komunikace mezi NPC není v implementaci řešena přímým předáváním informací o detekovaných hrozbách. Namísto toho je zaměřena na sdílení znalostí o herních předmětech, především o jejich výchozích pozicích a případném nesprávném umístění.

Základním prvkem tohoto systému je třída `BP_GameStateBase`, která udržuje společný seznam předmětů ve scéně. Každý předmět odvozený od třídy `BP_BaseItem` se při spuštění zaregistruje do tohoto seznamu pomocí funkce `AddToItemList`. Současně je uložena jeho výchozí poloha prostřednictvím funkce `SetItemHomePosition`. Tím vzniká sdílený zdroj informací, ke kterému mohou NPC při svém chování přistupovat.

Pro zjištění předmětu, který je potřeba uklidit, slouží funkce `RequestItemCleanup`. Ta prochází registrované předměty a hledá takový, který je označen jako nesprávně umístěný. Pokud je takový předmět nalezen, vrací referenci na něj společně s jeho aktuální polohou.

Vlastní výměna informací mezi NPC je realizována v úloze `BTT_InteractWithOtherNPC`. Tato úloha vyhledá ostatní instance `BP_NPC` ve scéně a vybere jinou postavu než tu, která úlohu právě vykonává. Následně pracuje se seznamy známých předmětů jednotlivých NPC a porovnává informace o jejich umístění.

Pokud NPC získá od jiné postavy relevantní informaci o předmětu, může tuto informaci využít pro aktualizaci vlastních hodnot v *blackboardu*. V implementaci se tímto způsobem pracuje například s informací o ztraceném předmětu a s klíči souvisejícími s hledáním boudy. Díky tomu mohou NPC nepřímo sdílet informace o stavu herního světa, aniž by mezi nimi existoval samostatný systém zpráv nebo událostí.

5.7 Implementační problémy a rozhodnutí

Původní návrh systému komunikace mezi NPC počítal s individuální komponentou `BPC_MemorySystem` připojenou ke každé NPC postavě. Tato komponenta měla uchovávat záznamy o objektech, se kterými postava přišla do kontaktu, a umožňovat přímou výměnu těchto záznamů mezi NPC.

Při implementaci se však ukázalo, že synchronizace paměťových záznamů mezi instancemi komponent napříč různými postavami nebyla spolehlivá. Docházelo k nekonzistentním stavům, kdy si NPC předávaly neaktuální nebo neplatné reference.

Problém byl vyřešen přechodem na centralizovaný přístup prostřednictvím třídy `BP_GameStateBase`. Místo individuálních paměťových komponent byl zaveden společný registr objektů dostupný všem NPC. Každý objekt si při inicializaci zaregistruje výchozí polohu, ke které NPC přistupují při rozhodování. Přímá výměna paměťových záznamů byla nahrazena porovnáním lokálního seznamu `ItemsKnownList` s tímto centrálním registrem.

Kapitola 6

Testování a evaluace řešení

Testování systému NPC probíhalo manuálně v rámci herní scény. Cílem testování bylo ověřit, zda implementovaná chování odpovídá návrhu a zda jednotlivé mechanismy fungují v očekávaných situacích.

Scénář	**Očekávané chování**	**Výsledek**
Přechody mezi stavy	NPC přechází mezi stavy `Passive`, `Investigating` a `Attacking`.	Ověřeno
Zraková percepce	Detekce hráče vede k přechodu do stavu `Attacking`.	Ověřeno
Ztráta vizuálního kontaktu	NPC se po ztrátě cíle vrací do pasivního chování.	Ověřeno
Sluchová percepce	Zvukový podnět vede k přechodu do stavu `Investigating`.	Ověřeno
Hlídkování NPC	NPC se pohybuje po přiřazené hlídkové trase.	Ověřeno
Změna hlídkové trasy	NPC po splnění podmínek změní aktuální trasu.	Ověřeno
Fázově řízené trasy	NPC upravují výběr tras podle fáze hry.	Ověřeno
Interakce s objekty	NPC dokáže interagovat s objekty (např. otevírání dveří).	Ověřeno
Sebrání předmětu	NPC dokáže sebrat interaktivní předmět (např. klíč).	Ověřeno
Položení předmětu	NPC po sebrání přesune předmět na jiné místo.	Ověřeno
Sdílení informací	NPC si předávají informace o předmětech.	Neověřeno

Tabulka 6.1: Přehled manuálně ověřených scénářů

Z tabulky 6.1 vyplývá, že základní mechanismy systému fungují dle očekávání. Byla ověřena zejména správná funkce percepce, přechodů mezi stavy a hlídkovacího chování NPC.

Naopak některé pokročilejší části systému nebyly v rámci testování spolehlivě ověřeny. Konkrétně se jedná o manipulaci s předměty po jejich sebrání a sdílení informací mezi NPC. Tyto části proto nelze považovat za plně otestované a představují omezení aktuální implementace.

Kapitola 7

Závěr

Cílem této bakalářské práce bylo navrhnout a implementovat systém chování nehráčských postav (NPC) v herním prostředí s využitím behaviorálních stromů v prostředí Unreal Engine 5.

Navržený systém kombinuje využití `AI Controlleru`, `Blackboardu` a behaviorálního stromu, což umožňuje modulární a přehlednou implementaci jednotlivých typů chování. Součástí řešení je také systém percepce založený na zrakových a sluchových podnětech a mechanismus sdílení informací o předmětech prostřednictvím `GameState`.

V rámci implementace se podařilo vytvořit funkční prototyp, ve kterém NPC dokáží hlídkovat, reagovat na hráče, interagovat s objekty ve scéně a pracovat s předměty. Chování jednotlivých postav odpovídá návrhu a je snadno rozšiřitelné o další prvky.

Testování ukázalo, že základní mechanismy systému fungují spolehlivě, zejména v oblasti percepce, přechodů mezi stavy a pohybu po hlídkových trasách. Naopak některé pokročilejší části, jako manipulace s předměty nebo sdílení informací mezi NPC, nebyly plně ověřeny a představují prostor pro další vylepšení.

7.1 Možná rozšíření a budoucí práce

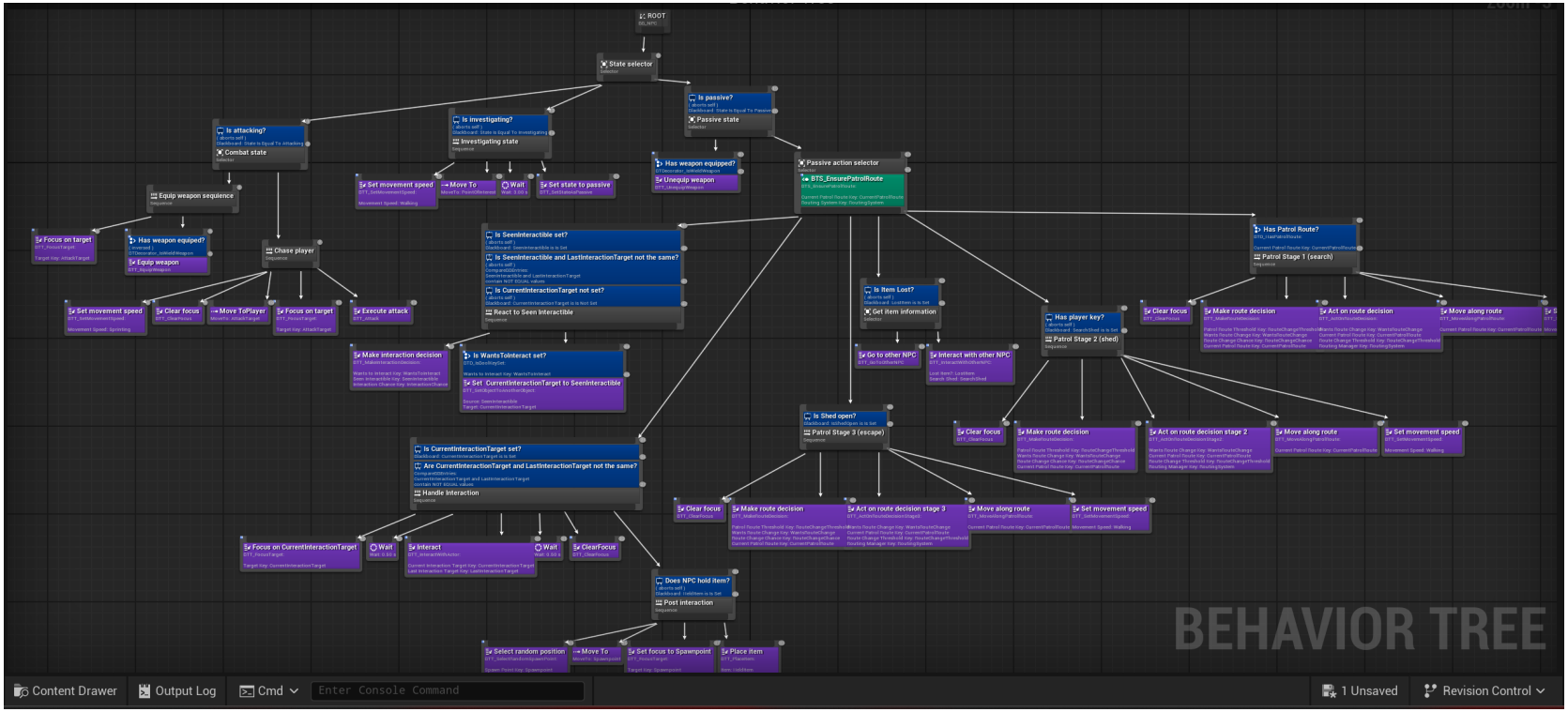
Navržený systém chování NPC by bylo možné dále rozšířit o reakce na dynamické změny prostředí. Jednou z možností je zohlednění denní doby a povětrnostních podmínek. Například v nočních hodinách by NPC mohly trávit více času v interiéru, zatímco během dne by se častěji pohybovaly v okolí domu. Podobně by bylo možné simulovat vliv deště, který by negativně ovlivňoval percepční schopnosti NPC, například zhoršením sluchu.

Dalším rozšířením by bylo zavedení interaktivních prvků v prostředí, jako jsou například světelné vypínače. NPC by mohly reagovat na změny osvětlení v místnostech, například snahou rozsvítit při vstupu do tmavého prostoru, nebo naopak vyhýbáním se neosvětleným místnostem. Takové chování by mohlo přispět k vyšší věrohodnosti a variabilitě simulace.

Uvedená rozšíření nebyla v rámci této práce implementována, je ale možné takových cílů dosáhnout při dalším vývoji této hry.

Literatura

1. COLLEDANCHISE, Michele; ÖGREN, Petter. *Behavior Trees in Robotics and AI: An Introduction*. Chapman a Hall/CRC, 2018.
2. BUTLER, Stuart; OLIVER, Tom. *Game Development Patterns with Unreal Engine 5: Build Maintainable and Scalable Systems with C++ and Blueprint*. Packt Publishing, 2023. ISBN 978-1-80324-325-2.
3. MILLINGTON, Ian. *Artificial Intelligence for Games*. San Francisco: Morgan Kaufmann, 2006. The Morgan Kaufmann Series in Interactive 3D Technology. ISBN 978-0-12-497782-2.
4. GREGORY, Jason. *Game Engine Architecture*. 3rd. CRC Press, 2018.
5. EPIC GAMES. *Gameplay Systems in Unreal Engine* [`https://dev.epicgames.com/documentation/unreal-engine/gameplay-systems-in-unreal-engine`]. 2025. Accessed: 2025-04-27.
6. EPIC GAMES. *Pawn* [`https://dev.epicgames.com/documentation/unreal-engine/pawn-in-unreal-engine?application_version=5.0`]. 2026. Accessed: 2026-04-27.
7. EPIC GAMES. *AI Controllers in Unreal Engine* [`https://dev.epicgames.com/documentation/unreal-engine/controllers-in-unreal-engine?application_version=5.0`]. 2026. Accessed: 2026-04-27.
8. EPIC GAMES. *Behavior Trees in Unreal Engine* [`https://dev.epicgames.com/documentation/en-us/unreal-engine/behavior-trees-in-unreal-engine?application_version=5.0`]. 2026. Accessed: 2026-04-27.
9. EPIC GAMES. *Navigation System* [`https://dev.epicgames.com/documentation/unreal-engine/navigation-system-in-unreal-engine`]. 2026. Accessed: 2026-04-27.
10. EPIC GAMES. *AI Perception in Unreal Engine* [`https://dev.epicgames.com/documentation/en-us/unreal-engine/ai-perception-in-unreal-engine?application_version=5.0`]. 2026. Accessed: 2026-04-27.



Obrázek 1: Kompletní struktura behaviorálního stromu použitého v systému NPC

Příloha A

Struktura projektu

Projekt byl verzován pomocí verzovacího systému Git.

Repozitář je dostupný online na adrese `https://github.com/bad-account/bachelor_thesis`.

```
/
├── BachelorThesis ..... LaTeX projekt
│   └── BachelorThesis.pdf ..... PDF soubor s textem práce
├── Config ..... konfigurace projektu
├── Content ..... herní assety
├── videos ..... záznamy z testování
├── Descent_into_Despair.uproject ..... Unreal Engine Project File
└── README.md ..... popis repozitáře
```